Formula 1 Podium Prediction Companion

Milestone 1: Data Engineering, Modelling and Explainability: Analytical Report

Team members: _____

Kaggle notebook: _____ GitHub repository: _____

	Summary
Task	Binary classification: will a driver finish on the podium (positionOrder ≤ 3)? Prediction cutoff: after qualifying, before the start.
Data	Ergast / Kaggle F1 World Championship 1950–2024: 14 relational tables. Modelling table: 8,970 unique driver-races (2003–2024), 28 leakage-safe features.
Split	Temporal: train 2003–2016 · validation 2017–2020 · unseen test 2021–2024.
Selected model	Random forest (chosen on validation PR-AUC): test PR-AUC 0.720, ROC-AUC 0.940, F1 0.677. Grid-only baseline: 0.646 / 0.911 / 0.660.
Q1	Front row → podium: 68% (2003–13) vs 76% (2014–24). The circuit ranking is not stable across eras (Spearman ρ = 0.30).
Q2	No home advantage after controlling for grid position (logistic OR 0.92, 95% CI 0.78–1.08).
Q3	Mechanical DNF rate 9.6% → 6.0% of starts. Worst: Caterham, Lotus, Toro Rosso (2014–21); Alfa Romeo, Alpine, AlphaTauri (2022–24).

1. Data audit and cleaning

All 14 CSVs were read as untyped text so that the audit sees the files exactly as written. The raw files are never modified; every step works on copies. The checks are programmatic, not manual.

- **Sentinels.** The only missing-value token is the literal *\N*, found in 40 columns (e.g. 41% of *results.position*, 71% of race *time*, 64% of *qualifying.q3*). Before cleaning, pandas reported 0% missing. All *\N* and empty strings became NaN, and padded text was stripped (e.g. the nationality "Argentinian ").
- **Semantic typing.** Keys became nullable integers, measures floats, dates datetime64. Durations (lap times, Q1–Q3, pit stops, fastest laps) were parsed from *h:mm:ss.sss* into seconds. The parser was verified against the independent millisecond columns (max error 1 ms on 839k laps).
- **Keys.** All surrogate primary keys are unique and non-null. All 23 single-column foreign keys and 3 composite references resolve: 0 orphans after the season-scope filter (seasons ≤ 2024). Two problems were found: 1,979 duplicated *lap_times* keys (a block of 1988–89 laps loaded twice, kept once) and two spellings of the US in *circuits.country* (canonicalised).
- **Grain.** 176 result rows form 85 repeated (raceId, driverId) pairs, all 1950–1978 shared drives. Rule: pre-race attributes come from the car the driver *started* (lowest valid grid); the outcome is the driver's *best* classified result (official records credit shared-drive podiums). Result: 26,759 → **26,668 unique driver-races**. 16 podium labels differ from a naive 'keep first row' rule.
- **Coverage drift.** Qualifying starts in 1994, is patchy 1996–2002, and is complete only from 2003. Modelling therefore uses 2003–2024, and Section 6 confirms this empirically.

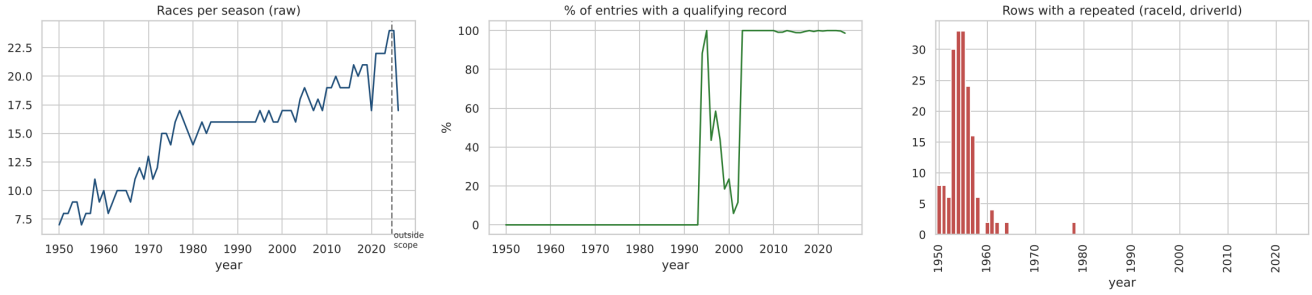


Figure 1: Raw data: races per season, qualifying coverage (complete only from 2003) and duplicate (raceId, driverId) rows (1950s shared drives).

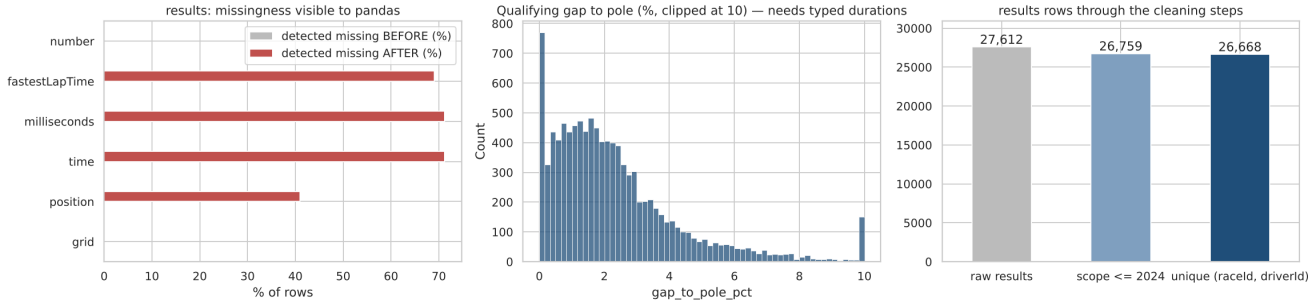


Figure 2: Effect of cleaning: missingness visible to pandas before/after, qualifying gap to pole (only possible after typing durations), and the row funnel.

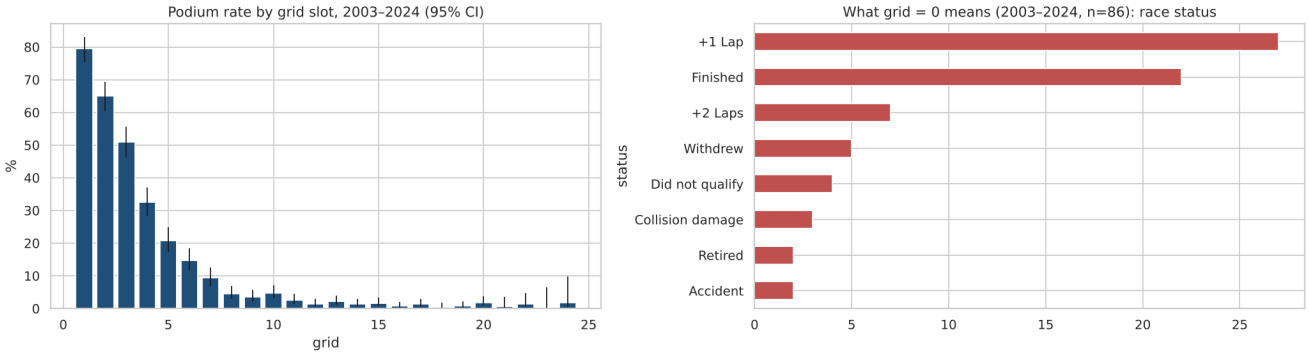


Figure 3: After cleaning: podium rate by grid slot (95% CI), and what grid = 0 really means (pit-lane starts, DNS).

Key takeaway: cleaning exposes the true missingness, types every column, verifies every key and fixes the grain to one row per driver per race.

2. Data-engineering questions

Q1. Which circuits convert a front-row start into a podium most often, and did this change in the hybrid era?

Joins: results + qualifying + races + circuits. **Grain:** driver-race. **Window:** 2003–2013 vs 2014–2024 (complete qualifying data). **Minimum:** 10 front-row starts per circuit and era. Wilson 95% intervals.

- A front-row **grid** start became a podium 68% of the time pre-hybrid and 76% in the hybrid era (dominant teams).
- Top circuits pre-2014: Magny-Cours (11/12), Istanbul, Monaco, Monza. Hybrid era: Suzuka (89%), Yas Marina (86%), Monaco (85%), Spa and Barcelona (82%).
- **The ranking changes:** Spearman ρ = 0.30 (p = 0.29) across the 14 circuits ranked in both eras. Only Monaco stays near the top.
- Qualifying and grid front rows disagree in 74 driver-races (penalties, pit-lane starts). Grid-based conversion is slightly higher, so the **grid** is the better pre-race signal. This motivates the *grid_penalty* feature.

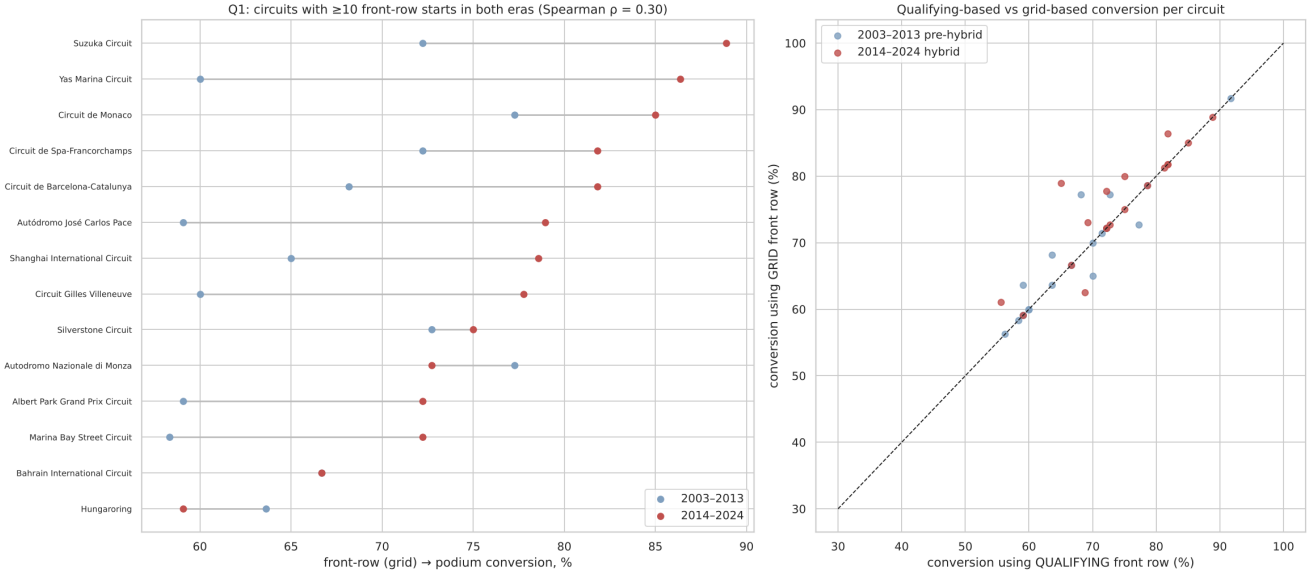


Figure 4: Q1: conversion per circuit in both eras (left), and qualifying-based vs grid-based conversion (right).

Q2. Do drivers podium more often at home, after controlling for starting position?

Joins: results + drivers (nationality) + races + circuits (country). Nationalities are mapped to countries with an explicit dictionary (dual nationalities count both). **Control:** grid bands P1, P2–3, P4–6, P7–10, P11+, using a Cochran–Mantel–Haenszel test and a logistic regression with grid band and decade. Grid-0 rows are excluded.

- Raw: home 11.4% vs away 13.8%, but home drivers start further back (mean grid 13.3 vs 11.7).
- Controlled: CMH OR = 0.94 (p = 0.42); logistic OR = 0.92 [0.78, 1.08]. In 2003–2024: OR = 0.76 [0.51, 1.13].
- **Answer: no evidence of a home advantage** once starting position is accounted for. The raw gap reflects where home drivers start. Caveat: nationality is not the same as residence, and the home sample at the front of the grid is small.

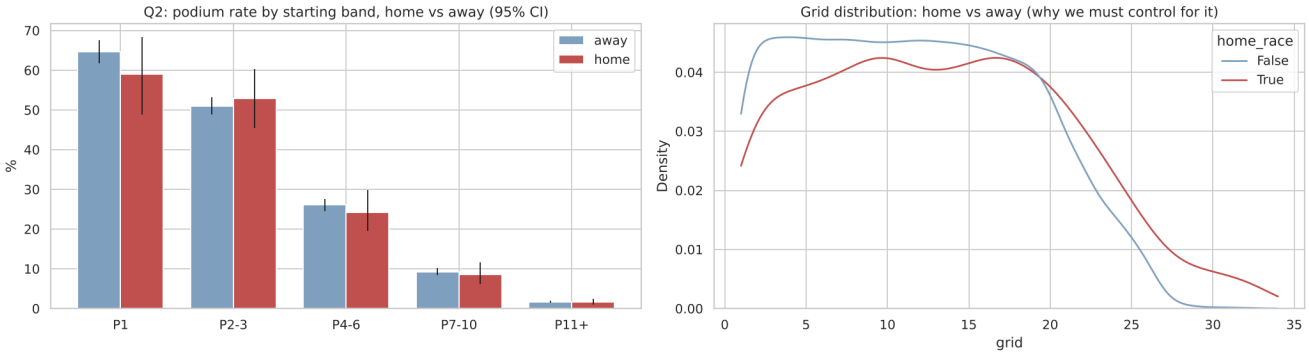


Figure 5: Q2: podium rate by starting band, home vs away (95% CI), and the different grid distributions.

Q3. Which constructors have the highest mechanical-retirement rate, 2014–2021 vs 2022–2024?

Joins: results + status + constructors + races. **Definition:** a race start ending with a component-failure status (engine, gearbox, power unit, ERS, hydraulics, brakes, suspension, electrics, ...). Excluded: crashes and collisions, driver causes, regulation decisions, non-starters. Minimum 30 starts. The uninformative "Retired" code is shown as an upper bound.

- Overall: 9.6% of starts (2014–2021) → 6.0% (2022–2024).
- 2014–2021 worst: Caterham 23.5%, Lotus F1 17.3%, Toro Rosso 16.1%, McLaren 14.5%, Renault 14.0%. Best: Mercedes 3.8%.
- 2022–2024 worst: Alfa Romeo 12.5%, Alpine F1 Team 11.0%, AlphaTauri 8.0%. Best: Red Bull and McLaren 3.7%. McLaren went from 4th-worst to among the best.

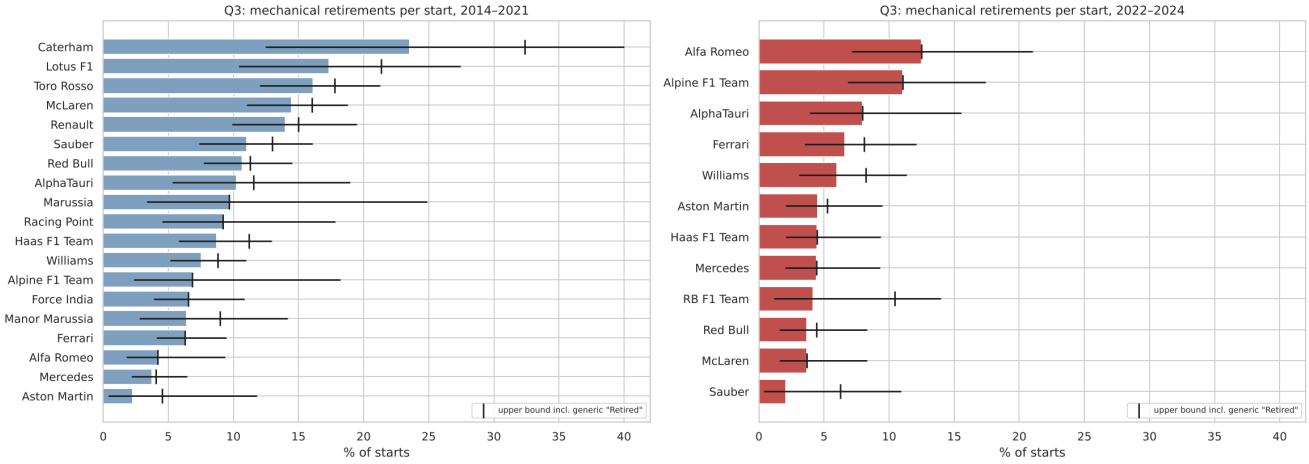


Figure 6: Q3: mechanical retirements per start with 95% CI; tick = upper bound if every generic "Retired" were mechanical.

3. Features, leakage control and split

- **Allowed:** grid (published before the start), qualifying, static attributes, and history from *earlier* races. **Forbidden:** anything produced by the race itself (result, points, laps, status, pit stops, lap times, fastest laps), and also the standings of the same race, which are computed afterwards.
- **Implementation:** every history feature is computed on date-sorted data with *shift(1)* before any rolling or expanding window. Standings come from the previous round. Constructor form is aggregated per race first, so the team-mate's current result never leaks.
- **Automated leakage test:** scrambling the outcome of a 2022 race and deleting its standings leaves that race's features identical, while the next race's features change (test passes). No feature has |Spearman| ≥ 0.9 with the target.
- **Split:** temporal by season: train 2003–2016 (5,591 rows), validation 2017–2020 (1,580), test 2021–2024 (1,799); podium rate 14–15% in each. A random split would mix rows of the same race and use future form, so the forward split mimics real use and includes the 2022 regulation change.
- **Preprocessing (fitted on train only):** winsorising extreme qualifying gaps at the 1st/99th percentile, median imputation plus missing indicators (structural missingness such as rookies or round 1), and standardisation for the logistic regression and the NN.

Selected features and why each is valid at the cutoff

group	feature	why it is valid before the race	univariate AUC (train)
grid	grid_pos	official starting grid (after penalties) is published before the start; grid 0 → back of field	0.904
grid	pitlane_start	known from the published grid	0.501
qualifying	quali_pos	qualifying session result (Saturday)	0.907
qualifying	quali_gap_pct	computed from the qualifying lap times of this weekend	0.886
qualifying	grid_penalty	grid − qualifying position; both known before the start	0.552
qualifying	teammate_quali_diff	both drivers' qualifying positions are known after qualifying	0.602
standings	drv_pts_share	driver standings after the PREVIOUS round, divided by the leader	0.848
standings	drv_champ_pos	championship position after the previous round	0.886
standings	drv_season_wins	wins so far this season, before this race	0.745
standings	con_pts_share	constructor standings after the previous round	0.852
standings	con_champ_pos	constructor position after the previous round	0.855
standings	prev_season_drv_pos	final position in the previous season	0.809
standings	prev_season_con_pos	final constructor position in the previous season	0.812
standings	first_round	calendar fact (no standings yet)	0.5
driver_form	drv_podium_rate_5	last 5 races of the driver, shift(1): excludes this race	0.845
driver_form	drv_avg_finish_5	last 5 races, shift(1)	0.855
driver_form	drv_dnf_rate_5	last 5 races, shift(1)	0.625
driver_form	drv_avg_grid_5	last 5 races, shift(1)	0.872
constructor_form	con_podium_rate_5	team results aggregated per race, then the last 5 races, shift(1) (team-mate's current result excluded)	0.865
constructor_form	con_best_finish_5	as above	0.879
constructor_form	con_mech_dnf_rate_10	team mechanical DNFs in the last 10 races, shift(1)	0.647
experience	drv_prior_starts	count of earlier starts	0.678
experience	drv_career_podium_rate	expanding mean of earlier races, shift(1)	0.817
experience	drv_circuit_starts	earlier starts at this circuit	0.659

group	feature	why it is valid before the race	univariate AUC (train)
experience	drv_circuit_podium_rate	earlier podium rate at this circuit, shift(1)	0.689
experience	driver_age	date of birth and race date (static)	0.553
context	home_race	nationality and circuit country (static)	0.502
context	field_size	number of entrants on the grid	0.523

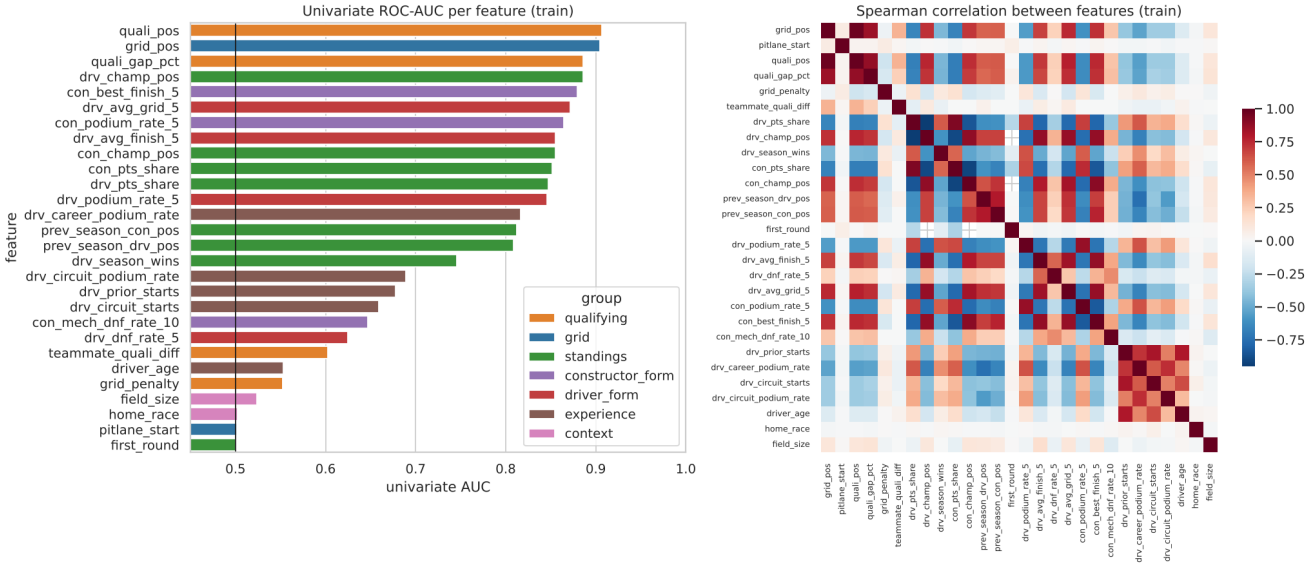


Figure 7: Feature-selection evidence on the training split: univariate ROC-AUC per feature, and the Spearman correlation between features.

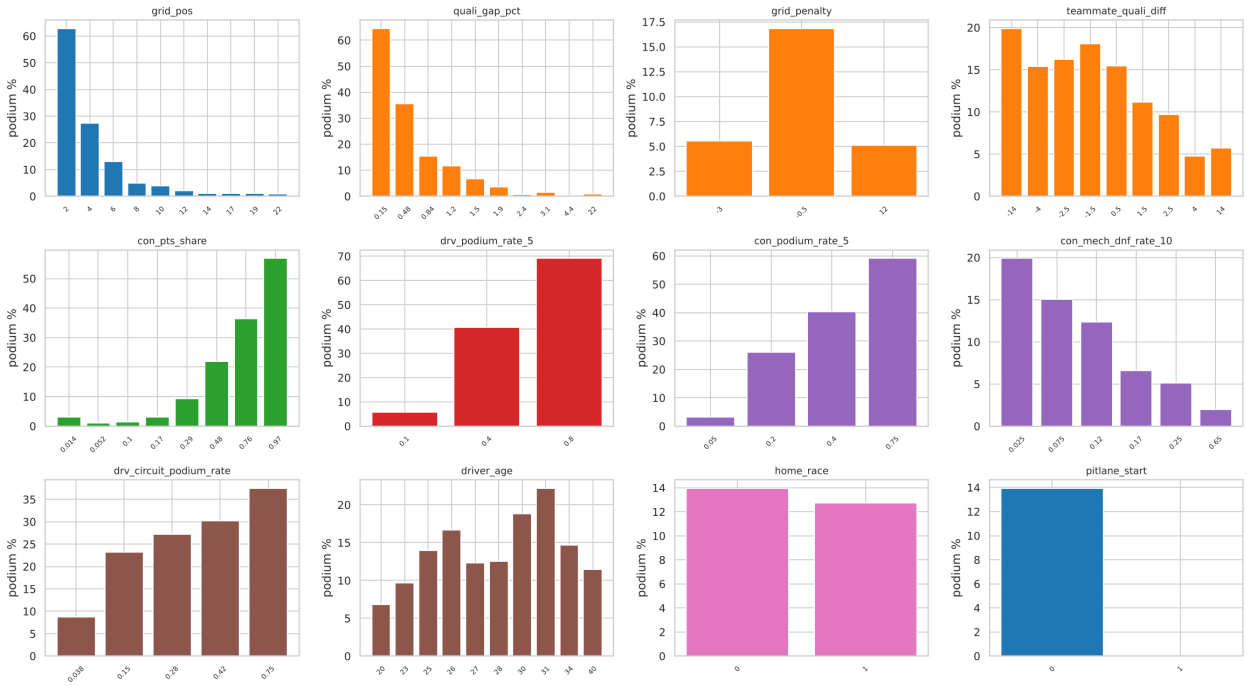


Figure 8: Podium rate across feature bins (training data).

Key takeaway: grid and qualifying (AUC ≈ 0.90) and car strength (standings, constructor form ≈ 0.85) carry the signal. Home race and field size are weak and were kept only to be tested by the ablation.

4. Models and evaluation

- **Primary metric: PR-AUC.** Podiums are only 14–15% of rows, so ROC-AUC is inflated by easy back-markers. PR-AUC measures how well the real contenders are ranked and is threshold-free, which makes it the selection metric. **Secondary: F1** at the threshold that maximises F1 on *validation*: the quality of the yes/no call.
- **Attempts:** grid-rule heuristic (baseline 0), logistic regression (statistical baseline, 8 trials), random forest (9), LightGBM (12) and a shallow NN with 1–2 ReLU layers, Adam, L2 and early stopping on validation (6). That is 36 trials in total, all kept in the notebook. Every model uses identical rows and features. Hyper-parameters and thresholds were chosen on validation only.

model	split	ROC-AUC	PR-AUC	F1	Precision	Recall	top-3 hit rate
Baseline 0: grid rule	val	0.926	0.685	0.715	0.716	0.713	0.717
Baseline 0: grid rule	test	0.911	0.646	0.66	0.662	0.659	0.663
Logistic regression (baseline)	val	0.941	0.708	0.723	0.64	0.831	0.671
Logistic regression (baseline)	test	0.937	0.712	0.668	0.632	0.707	0.656
Random forest	val	0.944	0.728	0.721	0.683	0.764	0.713
Random forest	test	0.94	0.72	0.677	0.687	0.667	0.663
LightGBM	val	0.945	0.722	0.728	0.655	0.819	0.705
LightGBM	test	0.938	0.724	0.686	0.633	0.748	0.667
Shallow NN	val	0.942	0.714	0.718	0.625	0.844	0.696
Shallow NN	test	0.93	0.694	0.653	0.62	0.689	0.644

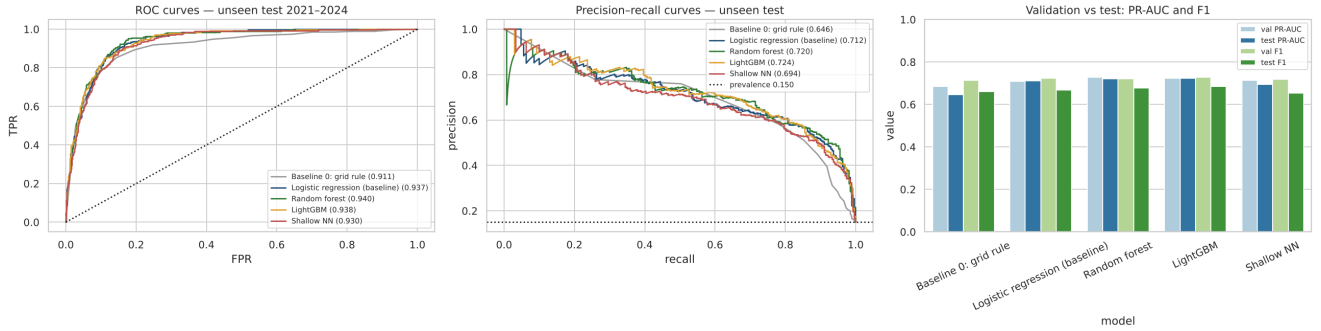


Figure 9: ROC and precision-recall curves on the unseen test seasons, and validation vs test PR-AUC/F1.

- All learned models beat the grid-only baseline on test PR-AUC. The Random forest was selected (validation PR-AUC 0.728) and scores 0.720 on test. LightGBM (0.724) and logistic regression (0.712) are close, and the NN (0.694) is slightly behind (small data for a neural network).
- The tree ensembles overfit on train (train PR-AUC 0.83–0.85), but validation and test agree, so they generalise to the 2022+ regulations. Season-level PR-AUC ranges from 0.67 to 0.78.
- Honest limit: the per-race top-3 hit rate (about 0.66 on test) matches the grid rule. The model is better at estimating podium likelihood across all drivers than at re-ordering the front of one grid.
- **Limitations per family:** logistic regression captures only additive, monotone effects. The forest and LightGBM can memorise and their probabilities are less smooth. The NN needs scaling and imputation, depends on the seed, and is the least interpretable.

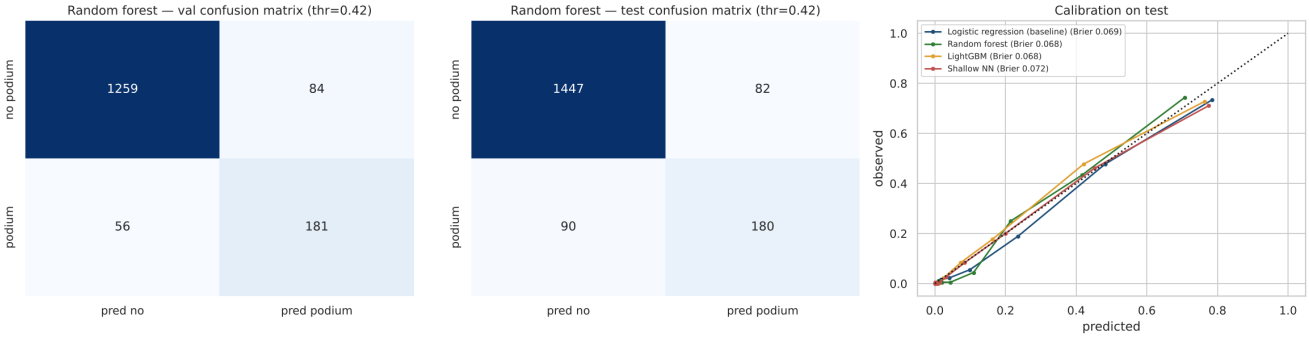


Figure 10: Selected model: confusion matrices (validation/test, threshold from validation) and calibration on test.

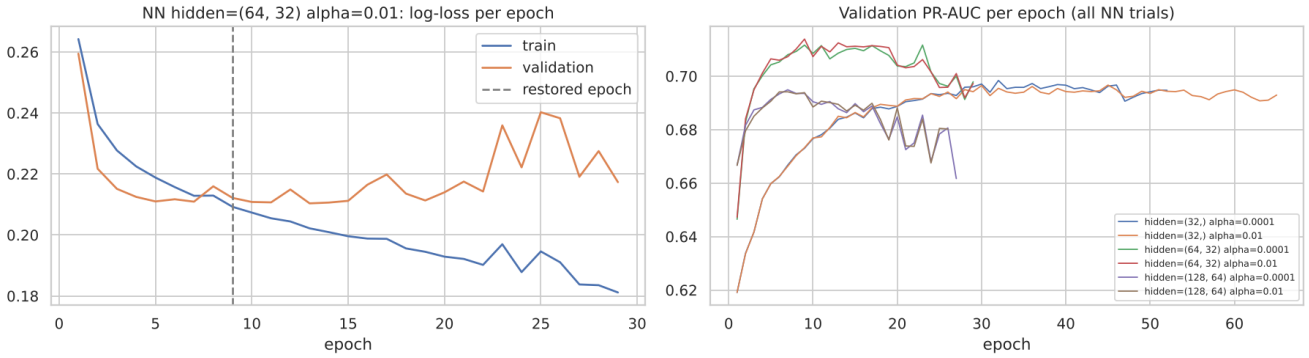


Figure 11: Shallow NN: train/validation loss per epoch with the restored early-stopping epoch, and validation PR-AUC for all NN trials.

5. Experiments

5.1 Feature-group ablation (one group removed at a time, retrained)

model	removed group	n features	val PR-AUC	test PR-AUC	Δ val PR-AUC	Δ test PR-AUC
Random forest	(none — full model)	28	0.7275	0.72	0.0	0.0
Random forest	grid	26	0.7136	0.7052	-0.0139	-0.0148
Random forest	qualifying	24	0.7193	0.7209	-0.0082	0.0009
Random forest	standings	20	0.7082	0.7121	-0.0193	-0.0079
Random forest	driver_form	24	0.7326	0.7183	0.0051	-0.0017
Random forest	constructor_form	25	0.729	0.7132	0.0015	-0.0068
Random forest	experience	23	0.7291	0.7233	0.0015	0.0032
Random forest	context	26	0.7279	0.7193	0.0004	-0.0007
Logistic regression (baseline)	(none — full model)	28	0.7082	0.712	0.0	0.0
Logistic regression (baseline)	grid	26	0.7084	0.7122	0.0002	0.0002
Logistic regression (baseline)	qualifying	24	0.7059	0.7023	-0.0023	-0.0097
Logistic regression (baseline)	standings	20	0.6999	0.7116	-0.0083	-0.0005
Logistic regression (baseline)	driver_form	24	0.7113	0.7151	0.0031	0.003
Logistic regression (baseline)	constructor_form	25	0.7099	0.7138	0.0017	0.0017
Logistic regression (baseline)	experience	23	0.7058	0.7162	-0.0024	0.0041
Logistic regression (baseline)	context	26	0.7074	0.713	-0.0008	0.001

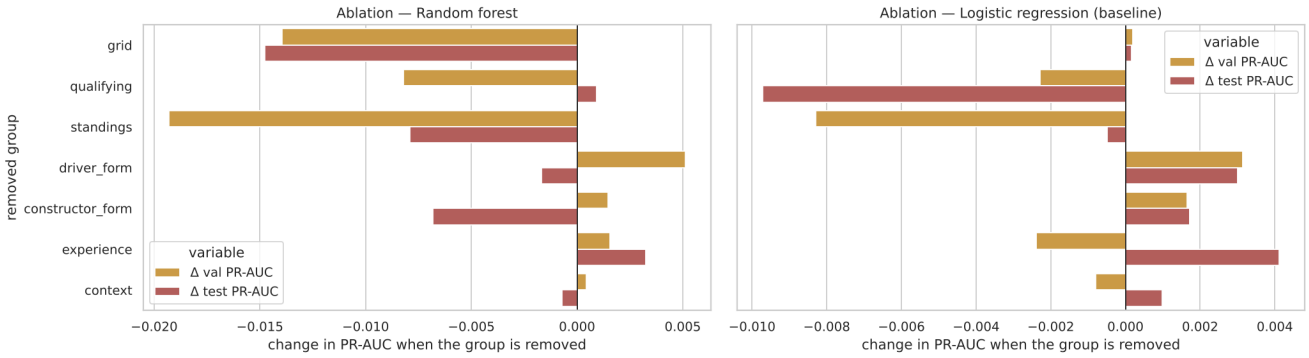


Figure 12: Change in PR-AUC when each group is removed (negative = the group helps).

- For the random forest, removing **standings** or **grid** hurts most. The logistic model does not need grid, because *quali_pos* carries the same information.
- Removing grid **and** qualifying together drops validation PR-AUC to 0.621: the two groups are substitutes. Grid and qualifying alone (6 features) reach 0.704.
- Driver form, constructor form, experience and context change PR-AUC by no more than about ±0.007, within run-to-run noise.

5.2 Preprocessing-order experiments (logistic regression)

experiment	variant	val PR-AUC	test PR-AUC	val F1	test F1
P1 fit-before/after split	fit on TRAIN only (correct)	0.7082	0.712	0.7229	0.6678
P1 fit-before/after split	fit on ALL seasons (leaky)	0.7074	0.7114	0.7256	0.6643

experiment	variant	val PR-AUC	test PR-AUC	val F1	test F1
P2 impute/scale order	impute 0 → scale	0.7057	0.7075	0.7206	0.6678
P2 impute/scale order	scale → impute 0 (= mean)	0.706	0.7088	0.7216	0.6667
P3 feature-selection order	select on TRAIN only (correct)	0.6914	0.7105	0.7269	0.6689
P3 feature-selection order	select on ALL rows incl. test labels (leaky)	0.6914	0.7105	0.7269	0.6689

- **P1** fitting the imputer and scaler on all seasons (leaky) vs train only: a tiny difference, because the statistics are similar. *drv_prior_starts* drifts (mean 90.7 → 104.0). It is still invalid in principle.
- **P2** impute 0 → scale vs scale → impute 0: a small difference, and both are worse than median plus indicator imputation (0.708). Imputing 0 before scaling creates impossible values such as "previous-season position 0".
- **P3** feature selection (mutual information, k = 12) on all rows including test labels vs train only: here the same 12 features are chosen. The leak would inflate test scores with weaker features, so selection belongs inside the training pipeline.

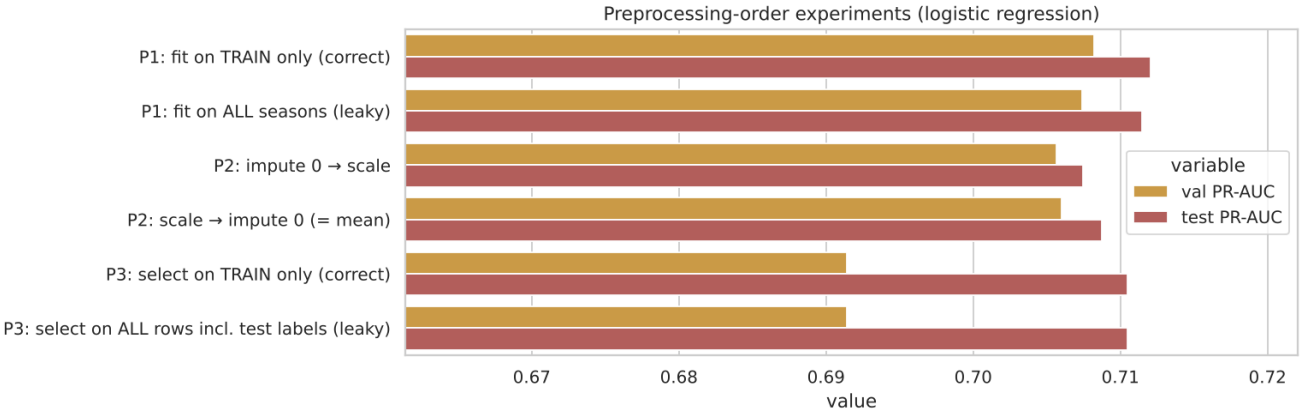


Figure 13: Preprocessing-order experiments.

5.3 Training window and feature-subset trials

Training on 1994–2016 (8,979 rows, 21% without qualifying data) vs 2003–2016: validation PR-AUC 0.724 vs 0.728, test 0.721 vs 0.720. Validation favours 2003+, which confirms the qualifying-coverage decision. Top-k feature subsets: 4 features already reach about 0.727, and k = 12–28 varies within ±0.005 (noise), so the full leakage-safe set is kept.

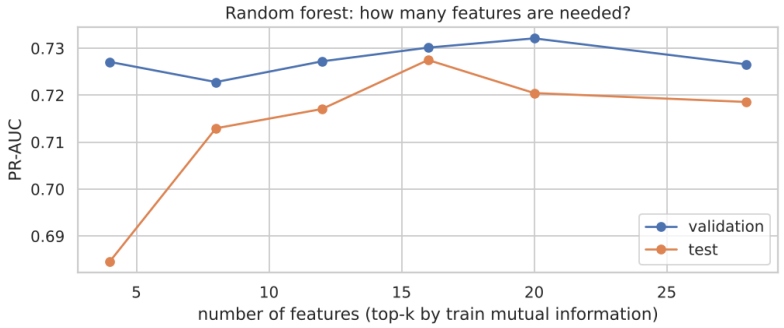


Figure 14: Validation/test PR-AUC against the number of features (ranked by training mutual information).

6. Explainability

Explanations are not causal. A feature being important means the model *relies on it to predict*, not that it *causes* podiums. Standings, for example, are proxies for car speed, the real (unobserved) driver of results. Correlated features (grid and qualifying) can share importance arbitrarily.

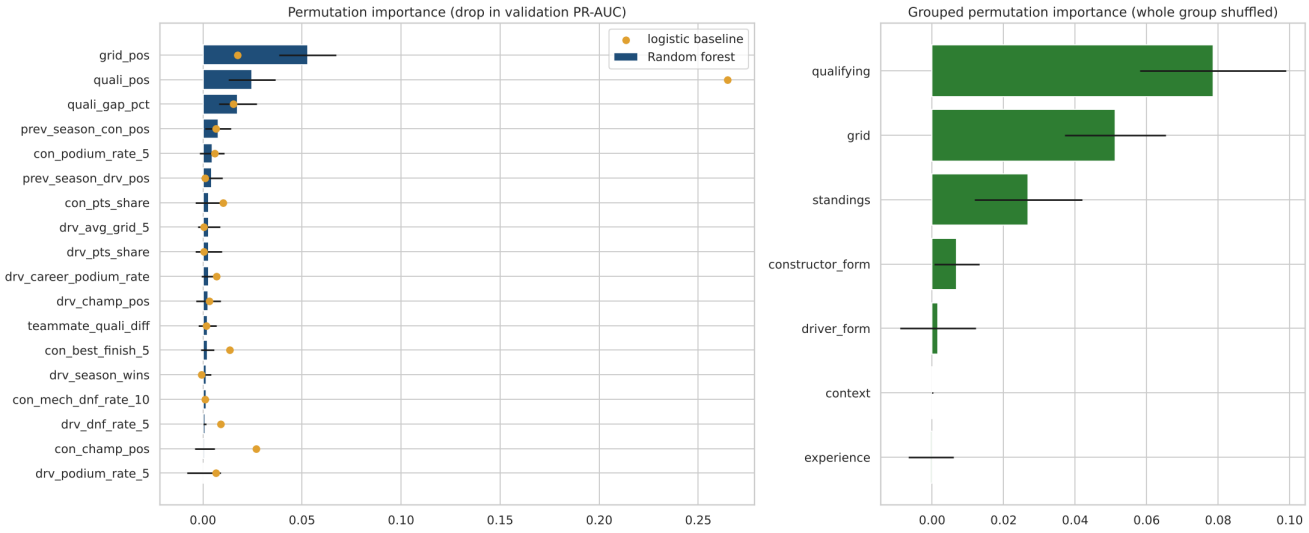


Figure 15: Permutation importance on validation (PR-AUC drop): single features (with the logistic baseline for comparison) and whole groups.

- Single features: *grid_pos* (0.053), *quali_pos*, *quali_gap_pct*, then previous-season constructor position and recent constructor podium rate. The logistic model relies mostly on *quali_pos*.
- Groups: qualifying 0.079, grid 0.051, standings 0.027; driver form, experience and context ≈ 0. This agrees with the ablation.

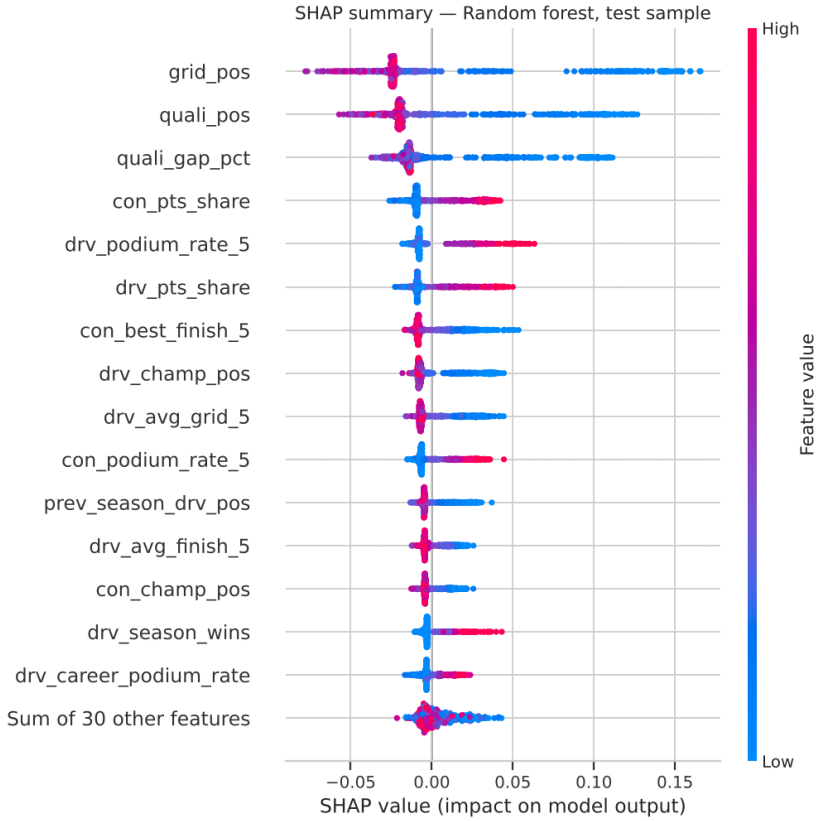


Figure 16: SHAP summary for the selected model on a test sample.

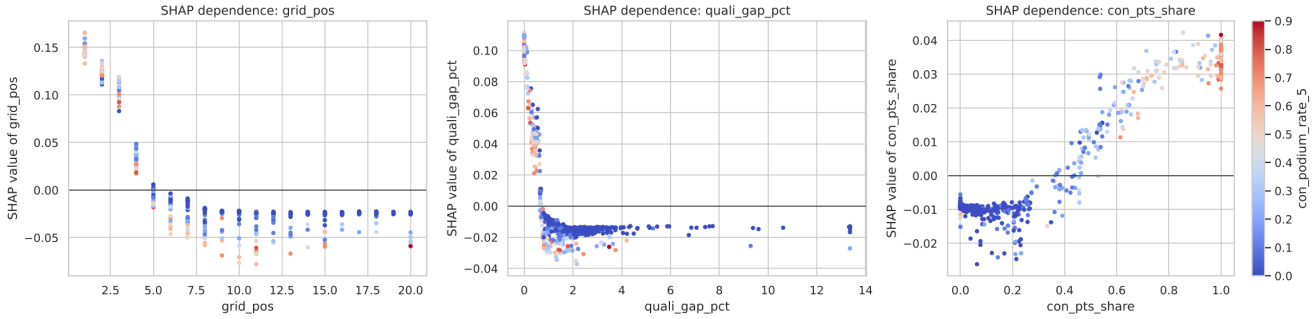


Figure 17: SHAP dependence: grid position, gap to pole, constructor points share.

- P1–P3 on the grid push the prediction up by +0.10 to +0.15. The effect collapses by P5 and is flat beyond P8.
- Being within about 0.5% of pole time pushes up by up to +0.10. Constructor points share raises the prediction steadily from about 0.3 to 1.0.
- SHAP and permutation importance agree on the top features.

Local explanation of one prediction

Max Verstappen, Belgian Grand Prix 2023: qualified P1 but started P6 after a gearbox penalty. Predicted P(podium) = 0.66, above the threshold. He finished P1. In the SHAP waterfall, pole in qualifying (+0.10) and zero gap to pole (+0.09) outweigh the grid slot (−0.06). LIME agrees: grid ≤ 6, qualifying ≤ 6 and gap ≤ 0.85% carry the largest weights, with surrogate $R^2 \approx 0.73$.

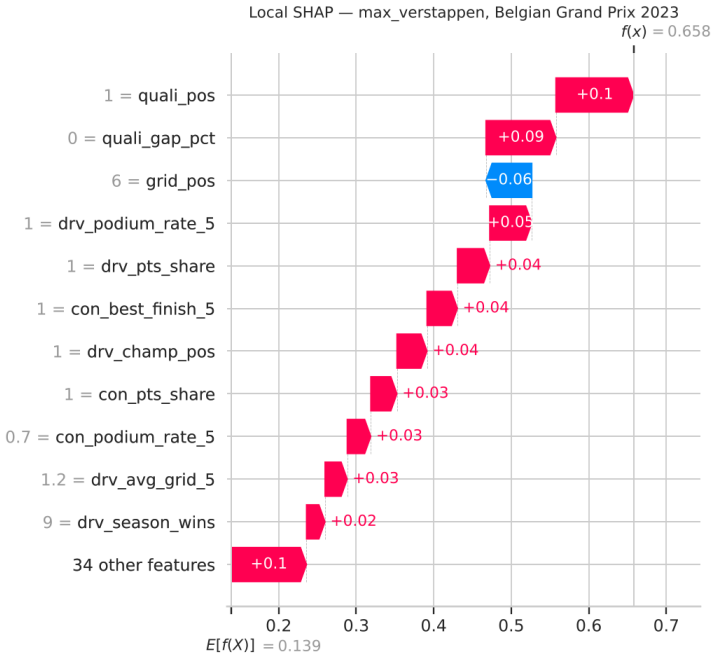


Figure 18: Local SHAP waterfall.

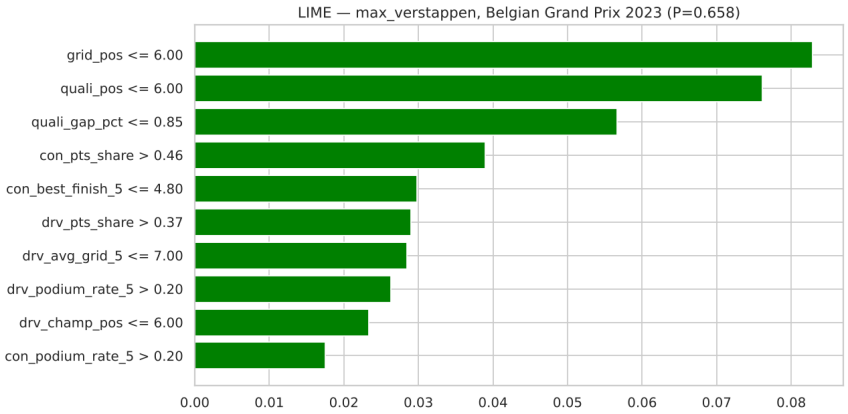


Figure 19: LIME explanation for the same row.

7. Inference function

- *predict_podium(record)* (in *f1podium/inference.py*) takes a plain dictionary of pre-race inputs, rejects post-race fields (e.g. *points*), derives *grid_pos* from *quali_pos* when needed, lets the pipeline impute the rest (training medians plus missing indicators), and reports which inputs were imputed.
- Complete record (Leclerc, Abu Dhabi 2024, all 28 inputs): P = 0.20, identical to batch scoring. He started P19 after a penalty and finished P3: an honest miss.
- Partially-missing record (5 of 28 inputs: qualifying P3, 0.25% gap, strong car): P = 0.38, with 20 inputs imputed and 3 derived. The same inputs from P15 give 0.06.

8. Known limitations

- **Qualifying-coverage drift:** qualifying is complete only from 2003, so earlier eras are used only as history.
- **Era sensitivity:** regulation changes and dominant-team periods shift base rates. Performance dips in 2022 and 2024. Retrain as seasons are added.
- **Small, dependent sample:** about 9k rows, 3 positives per race, team-mates sharing a car. Gaps of about 0.01 between top models are within season-to-season variation.
- **Within-race ordering:** the top-3 hit rate matches the grid rule. Race-relative features are the natural next step.
- **Unknowable events:** crashes, weather, safety cars and strategy happen after the cutoff.
- **Definitions:** home race uses nationality; the mechanical-failure list is a documented judgement; renamed constructors do not share history.
- **XAI:** importance describes the model, not causation.

9. Reproducibility and next milestones

The notebook runs top to bottom in about 3 minutes (Run All). It writes the reusable package *f1podium/* (cleaning, grain, entries, features, preprocessing, models, pipeline, inference) and saves *artifacts/* (the fitted model, *feature_spec.json* with features, threshold and metrics, the modelling table and the cleaned tables). *pipeline.build_modelling_table()* is verified to reproduce the modelling table exactly.